

République Algérienne Démocratique et Populaire
Université Abou Bekr Belkaid -Tlemcen

Faculté des Sciences

Département d'Informatique

Mémoire de Fin d'Études

Pour l'obtention du diplôme de Master en Informatique
Option : Réseaux et Systèmes Distribués (R.S.D)

Thème

Sécurité et contrôle d'accès dans les environnements IoT- Fog Computing

Réalisé par :

- TOUATI Anes Abdelmounaim
- ABOU BEKR Aymen

Soutenu le **23 Juin 2026** Devant le jury composé de :

GAOUAR Nihal
LEHSAINI Mohamed
OUDJEDI DAMERDJI Djennet
BRIKCI NIGASSA Amine

Présidente
Encadrant
Co-encadrante
Examineur

Année Universitaire : 2025/2026

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ
الحمد لله الذي بِنِعْمَتِهِ تَتِمُّ الصَّالِحَاتُ وَبِشْكْرِهِ تَدُومُ النِّعَمُ

Acknowledgements

First of all, I would like to thank Almighty God who gave me patience and strength to accomplish this work.

First and foremost, all praise and glory are due to Allah (SWT), whose guidance, mercy, and blessings have given me the strength and perseverance to complete this dissertation.

We would like to sincerely thank Mr. **LEHSAINI Mohamed** and Miss. **OUJEDI DAMERDJI Djennet**, who supervised this project. I am very grateful for their help, advice, and insightful comments.

We would like to thank all those who kept us in their prayers. I ask Allah to fill their homes with happiness and blessing.

Finally, our heartfelt thanks go to the members of the jury for their interest in this project, for agreeing to examine this work and enriching it with their experiences

Dédicaces

We wish to express our profound gratitude and sincere appreciation by
dedicating this final year project :

to our parents, whose sacrifices and unconditional love have been our
foundation.

May God grant them health and long life.

We also dedicate this work to our siblings, steadfast sources of strength
and encouragement.

To our extended family, we are grateful for their constant support despite
the distance.

Finally, to my partner and myself, for our shared commitment,
perseverance, and collaborative efforts that have made this achievement
possible.

TOUATI Anes Abdelmounaim & ABOU BEKR Aymen

Résumé

Ce projet de fin d'études présente la conception et le développement d'une plateforme sécurisée de partage de données IoT pour les environnements Fog/Edge Computing. Le système répond aux enjeux de contrôle d'accès et de confidentialité dans les architectures IoT à ressources limitées. Il repose sur un modèle de contrôle d'accès décentralisé basé sur les attributs, utilisant le chiffrement multi-autorités MA-ABE. L'approche consiste à transférer les opérations cryptographiques lourdes des nœuds IoT vers une passerelle Fog, qui gère le chiffrement MA-ABE des clés de session et les interactions avec une blockchain. Le système combine du chiffrement symétrique AES-128 pour les données et du chiffrement MA-ABE pour les clés de session, stockées sur Ethereum (Ganache) via un contrat Solidity. L'accès est conditionné par la satisfaction des politiques d'accès par les attributs de l'utilisateur. La simulation utilise Contiki-NG/Cooja, Python et Web3.py. Les résultats démontrent la faisabilité de cette approche pour IoT.

Keywords : IoT, MA-ABE, Chiffrement basé sur les attributs, Contrôle d'accès, Blockchain, AES, Fog computing, Contrat intelligent

Abstract

This project presents a secure IoT data-sharing platform for Fog/Edge environments, addressing access control and privacy in resource-constrained IoT systems. It uses a decentralized MA-ABE-based access control model (MaabeYJ14 with Charm-Crypto) to offload heavy cryptographic operations from IoT nodes to a Fog gateway, which handles MA-ABE encryption and blockchain interactions. The system combines AES-128 for sensor data and MA-ABE for session keys stored on Ethereum (Ganache) via a Solidity contract. Access is policy-based and attribute-driven. The simulation uses Contiki-NG/Cooja, Python, and Web3.py. Results demonstrate the approach's feasibility for IoT environments.

Keywords : IoT, MA-ABE, Attribute-Based Encryption, Access Control, Blockchain, AES, Fog Computing, Smart Contract

ملخص

يقدم مشروع التخرج هذا تصميم وتطوير منصة آمنة لمشاركة بيانات إنترنت الأشياء في بيئات الحوسبة الطرفية (Fog/Edge Computing). يستجيب النظام لتحديات التحكم في الوصول والسرية في بنى إنترنت الأشياء محدودة الموارد. يعتمد النظام على نموذج لامركزي للتحكم في الوصول قائم على السمات، باستخدام التشفير متعدد السلطات القائم على السمات (MA-ABE). يتمثل النهج في نقل العمليات التشفيرية الثقيلة من عقد إنترنت الأشياء إلى بوابة حوسبة طرفية (Fog)، التي تدير تشفير MA-ABE لمفاتيح الجلسة وتتفاعل مع سلسلة كتل (Blockchain). يجمع النظام بين التشفير المتماثل AES-128 للبيانات وتشفير MA-ABE لمفاتيح الجلسة، المخزنة على منصة إيثيريوم (Ganache) عبر عقد ذكي بلغة Solidity. يُشترط للوصول إلى البيانات استيفاء سمات المستخدم لسياسات الوصول المحددة. تستخدم المحاكاة أدوات Contiki-NG/Cooja و Python و Web3.py. تُظهر النتائج جدوى هذا النهج بالنسبة لإنترنت الأشياء.

الكلمات المفتاحية: إنترنت الأشياء، MA-ABE، التشفير القائم على السمات، التحكم في الوصول، Blockchain، AES، حوسبة الضباب، العقد الذكي.

TABLE DES MATIÈRES

General Introduction	1
1 IoT Security and Access Control	3
1.1 Introduction	3
1.2 IoT Overview	3
1.2.1 Definition and Concepts	3
1.2.2 IoT Environments	4
1.2.3 IoT Architecture	4
1.2.4 Resource Constraints	4
1.3 Computing Paradigms for IoT	5
1.3.1 Cloud Computing	5
1.3.2 Fog Computing	6
1.4 Fundamentals of IoT Security	6
1.4.1 Confidentiality	6
1.4.2 Integrity	6
1.4.3 Authentication and Authorization	6
1.4.4 Security Challenges in IoT	7
1.4.4.1 Attack Vectors on IoT Devices	7
1.4.4.2 Access Control Attacks	7
1.5 Access Control Mechanisms	7
1.5.1 DAC	7
1.5.2 MAC	8
1.5.3 RBAC	8
1.5.4 ABAC	8
1.5.5 Limitations of Existing Mechanisms	8
1.6 Attribute-Based Encryption	9
1.6.1 IBE	9
1.6.2 ABE	9

1.6.3	MA-ABE	9
1.6.4	Why MA-ABE for IoT Security	9
1.7	AES Symmetric Encryption	10
1.7.1	What is AES ?	10
1.7.2	AES-CBC Mode	10
1.7.3	Why AES is Suitable for IoT ?	10
1.8	Blockchain Fundamentals	11
1.8.1	What is Blockchain ?	11
1.8.2	Smart Contracts	11
1.8.3	Ethereum and Ganache	11
1.8.4	Blockchain for Secure Data Management	11
1.9	Conclusion	12
2	System Design and Architecture	13
2.1	Introduction	13
2.2	Objectives of the system designed	13
2.3	System Actors and Entities	14
2.3.1	IoT Node	14
2.3.2	Data Owner	14
2.3.3	Attribute Authority	14
2.3.4	Fog Node	14
2.3.5	Blockchain Ledger	14
2.3.6	Cloud Server	14
2.3.7	Data User	15
2.4	System diagrams	15
2.4.1	Use Case Diagram	15
2.4.2	Class diagram	16
2.5	Process workflow and protocols	17
2.5.1	Phase 1 : System initialization	17
2.5.2	Phase 2 : Data collection and storage	17
2.5.3	Phase 3 : Data decryption and Access	18
2.6	Smart contract design	18
2.7	Access control policy design	18
2.8	Conclusion	19
3	Development tools and environment	20
3.1	Introduction	20
3.2	Hardware and software environment	20
3.2.1	Development machine	20
3.2.2	Python environment	21
3.3	IoT simulation environment	21

3.3.1	Contiki-NG	21
3.3.2	Cooja Network Simulator	21
3.4	Blockchain Environment	22
3.4.1	Ganache	22
3.4.2	Solidity and smart contract	22
3.4.3	py-solc-x	23
3.4.4	Web3.py	23
3.5	Cryptographic environment	23
3.5.1	Charm-Crypto Library	23
3.5.2	PyCryptodome	24
3.6	System communication	24
3.6.1	Serial Socket - Cooja to Python	24
3.6.2	TCP sockets - Between Python components	25
3.7	Conclusion	25
4	Implementation and Testing	26
4.1	Introduction	26
4.2	Implementation of the system components	26
4.2.1	Phase 1 : System initialization	27
4.2.2	Phase 2 : Data collection and storage	29
4.2.3	Phase 3 : Data Decryption and Access	31
4.3	Scenario-Based Testing	32
4.3.1	Authorized User	32
4.3.2	Full End-to-End Test	32
4.4	Encountered Problems and Solutions	33
4.4.1	Blockchain Contract Redeployment	33
4.4.2	Cooja Serial Socket Connection	33
4.5	Conclusion	33
	General Conclusion	34
	Bibliographie	36

TABLE DES FIGURES

1.1	The 4-layer IoT architecture	5
2.1	Use case diagram	15
2.2	Class diagram	16
4.1	Smart Contract Deployment	27
4.2	Attribute Authority Terminal	27
4.3	Fog Node Terminal	28
4.4	Ganache terminal	29
4.5	Cooja network simulator	29
4.6	Data owner terminal	30
4.7	Cloud Server Terminal	31
4.8	Data User Terminal	32

LISTE DES TABLEAUX

2.1 Access control policies defined in the system. 19

ABAC Attribute-Based Access Control. i, 8

ABE Attribute-Based Encryption. i, 9

ACLs Access Control Lists. 7

AES Advanced Encryption Standard. 1, 3, 10

BLE Bluetooth Low Energy. 4

CBC Cipher Block Chaining. 10

DAC Discretionary Access Control. i, 7

GID unique global identifier. 9

GPP Global Public Parameters. 27

IBE Identity-Based Encryption. i, 9

ID unique identifier. 17, 28

IoT Internet of Things. 1, 3, 34

IV Initialization Vector. 10

MA-ABE Multi-Authority Attribute-Based Encryption. ii, 1, 3, 9, 34

MAC Mandatory Access Control. i, 8

NIST National Institute of Standards and Technology. 10

PK public keys. 17

PKI Public Key Infrastructure. 9

RBAC Role-Based Access Control. i, 8

SK private keys. 17

UML Unified Modeling Language. 15

Introduction

The rapid expansion of the Internet of Things has fundamentally transformed the way data is collected, transmitted, and consumed across modern environments. Connected sensors and embedded devices are now deployed at massive scale in domains such as healthcare, smart cities, industrial monitoring, and environmental sensing. These systems continuously generate sensitive data that must be protected against unauthorized access, tampering, and interception. However, the nature of Internet of Things (IoT) hardware characterized by limited processing power, constrained memory, and restricted energy budgets makes the application of traditional security mechanisms both impractical and insufficient.

Beyond hardware limitations, the distributed and multi-organizational nature of modern IoT deployments introduces a deeper architectural challenge : how can access to sensitive data be enforced in a decentralized manner, without relying on a single trusted authority that could become a bottleneck or a single point of failure.

This work addresses these challenges by proposing and implementing a secure IoT data sharing system that integrates three complementary technologies : Multi-Authority Attribute-Based Encryption (MA-ABE), Advanced Encryption Standard (AES) symmetric encryption, and Ethereum blockchain smart contracts. The proposed architecture enforces access control cryptographically, meaning that unauthorized users are cryptographically prevented from accessing the data, regardless of their position in the network. A hybrid encryption scheme is adopted to reconcile the computational demands of MA-ABE with the resource constraints of IoT hardware : raw sensor data is protected using lightweight AES-128-CBC encryption, while the key is secured using MA-ABE and anchored on an immutable blockchain ledger.

The system is implemented and validated in a controlled simulation environment using Contiki-NG and the Cooja network simulator for the IoT layer, Python for the middleware components, Ganache for the local blockchain, and Solidity for the smart contract.

This report is organized into four chapters :

- Chapter 1 presents IoT security and access control. It introduces the fundamental concepts

of IoT architectures, reviews existing access control mechanisms and their limitations, and details the cryptographic foundations, including AES encryption, Attribute-Based Encryption, and blockchain technology.

- Chapter 2 describes the architecture and design of the proposed system. It defines the roles of each actor in the system, details the encryption scheme, the access control policy design, the smart contract structure, and the complete workflow from data generation to authorized decryption.
- Chapter 3 presents the development environment and tools used to build the system. It covers the simulation platform (Contiki-NG and Cooja), the blockchain environment (Ganache and Solidity), the cryptographic library (Charm-Crypto), and the Python components used to implement each system actor.
- Chapter 4 details the practical implementation and testing of the system. It presents each component individually, traces the complete data flow end to end, and validates the system through scenario-based testing that confirms the correct enforcement of attribute-based access control.

Finally, the manuscript concludes with a section that recapitulates the development process of the proposed access control protocol.

CHAPITRE 1

IOT SECURITY AND ACCESS CONTROL

1.1 Introduction

The rapid growth of IoT technologies has transformed the way modern environments operate. Connected devices are now deployed to collect and transmit sensitive data in real time. However, this connectivity introduces significant security challenges, particularly in terms of data confidentiality. This chapter presents the fundamental concepts of IoT security, reviews existing access control mechanisms, and introduces the approaches relevant to this domain, namely AES symmetric encryption, decentralized blockchain technology, and MA-ABE.

1.2 IoT Overview

1.2.1 Definition and Concepts

The IoT refers to a network of interconnected physical objects capable of collecting, processing, and exchanging data through communication networks [1]. These objects, commonly known as IoT devices, include sensors, actuators, embedded systems, and smart appliances equipped with communication capabilities.

The concept of IoT extends the traditional Internet by enabling physical devices to interact with each other and with external systems without requiring continuous human intervention. Through the integration of sensing technologies, wireless communications, cloud computing, and data analytics, IoT systems provide real-time monitoring, automation, and intelligent decision-making capabilities [2, 3].

A typical IoT system consists of sensing devices that collect information from the physical environment, communication technologies that transmit the collected data, processing platforms that analyze and store the information, and applications that provide services to end users [4]. These systems are increasingly deployed in domains such as healthcare, transportation, agriculture, smart cities, and automation.

1.2.2 IoT Environments

IoT environments are characterized by the large-scale deployment of connected nodes across various application spaces. Whether in sensor networks or in data collection platforms, these configurations require seamless connectivity and continuous data flow.

These environments impose strict requirements regarding reliability, availability, and transmission security, as connected devices continuously capture and share metrics that directly influence software decisions and system monitoring.

1.2.3 IoT Architecture

IoT architectures are commonly organized into multiple layers, each responsible for a specific set of functions. Although several architectural models exist in the literature, the four-layer architecture remains one of the most widely adopted approaches [2] :

- The Perception Layer is the lowest layer of the architecture and interacts directly with the physical environment. It consists of sensors and actuators responsible for collecting data such as temperature, humidity, pressure, or motion, and performing actions based on received commands.
- The Network Layer is responsible for transmitting data collected by IoT devices to processing entities. This layer relies on communication technologies such as Wi-Fi, Bluetooth Low Energy (BLE), ZigBee, LoRaWAN, and traditional IP routing architectures [4].
- The Processing Layer, also known as the Middleware Layer, receives and processes data coming from the network layer. It performs functions such as data aggregation, protocol translation, temporary storage, security enforcement, and analytics. Cloud servers and intermediate nodes are commonly deployed within this layer.
- The Application Layer provides services to end users by transforming processed data into useful information. Examples include monitoring dashboards, predictive maintenance platforms, and environmental monitoring applications.

1.2.4 Resource Constraints

Implementing standard, robust security countermeasures on IoT devices is fundamentally restricted by the inherent resource limitations of the physical nodes [5]. These embedded microcontrollers are tightly constrained across several axes : computational capacity, memory availability, and energy reserves.

- Computational Limitations : Typical IoT peripheral nodes operate on low-power 8-bit or 16-bit microcontrollers with processing speeds measured in Megahertz (MHz) [6]. Heavy asymmetric cryptographic systems require intense modular exponentiation or scalar

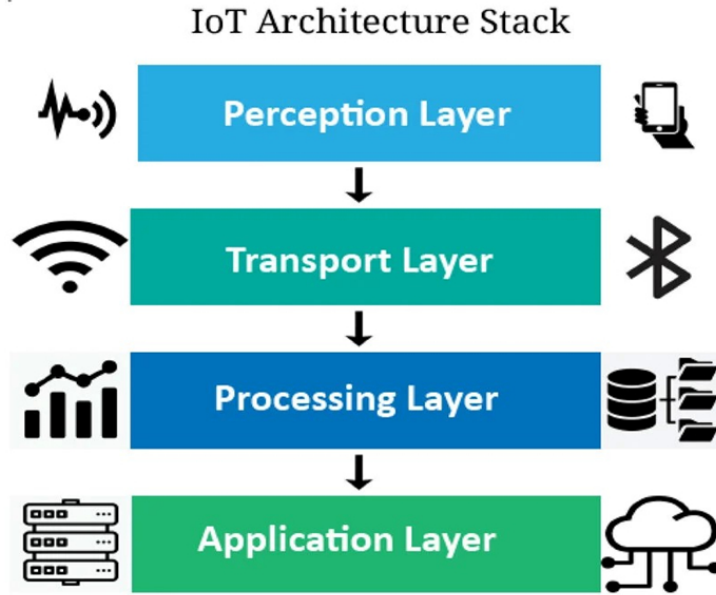


FIG. 1.1 : The 4-layer IoT architecture

multiplication operations that impose severe processing overhead, leading to unacceptable latency.

- **Memory Allocations :** Storage architectures on low-power nodes frequently restrict RAM to a few kilobytes and Flash memory to tens or hundreds of kilobytes [5, 7]. This restricted footprint prevents the integration of extensive security libraries or massive local access control matrices.
- **Energy Constraints :** Many IoT sensors rely on autonomous batteries designed to operate reliably for several years without human intervention [7, 8, 9]. Complex mathematical operations prolong the active CPU duty cycle and escalate radio transmission windows, which rapidly exhausts power reserves.

These constraints significantly influence the design of IoT security systems. Due to their limited computational power, memory, and energy resources, IoT devices often require lightweight security mechanisms and efficient cryptographic solutions that minimize resource consumption while maintaining an adequate level of protection [10].

1.3 Computing Paradigms for IoT

1.3.1 Cloud Computing

Cloud Computing constitutes a fundamental layer for long-term data storage, advanced analytics, and global accessibility [8]. It offers virtually unlimited computational resources, scalable storage solutions, and comprehensive hosting platforms.

In decentralized IoT monitoring, data generated at the edge and pre-processed at the fog layer is securely forwarded to a Cloud Server [11]. Cloud platforms can securely store large volumes of IoT data while providing scalability and high availability.

1.3.2 Fog Computing

As IoT networks scale, relying exclusively on centralized cloud platforms introduces significant challenges regarding network congestion, propagation latency, and bandwidth consumption [12]. To address these architectural bottlenecks, Fog Computing introduces an intermediate, decentralized computing layer distributed geographically between edge devices and centralized data centers.

In IoT environments, a Fog Node acts as a localized intelligence hub situated close to the physical IoT sensors [13]. By handling localized tasks, the fog layer reduces the burden on resource-constrained endpoint sensors and enables low-latency processing closer to the data source.

Fog computing is particularly relevant in security-sensitive IoT deployments, where cryptographic operations such as key management, encryption, and access policy enforcement can be handled at the fog layer rather than burdening the constrained edge devices [12, 13].

1.4 Fundamentals of IoT Security

1.4.1 Confidentiality

Confidentiality ensures that sensitive data remains completely inaccessible to unauthorized individuals or entities, or processes during both transmission and storage phases [14]. Within an IoT context, maintaining confidentiality prevents malicious actors from gaining access to sensitive data or real-time operational status updates.

1.4.2 Integrity

Integrity guarantees that data has not been altered, manipulated, or corrupted by unauthorized individuals, entities during its traversal across the network infrastructure [14]. In critical IoT processes, unverified data modification could result in system malfunctions.

1.4.3 Authentication and Authorization

Authentication verifies the identity of a user or system entity trying to access data, while Authorization maps out their explicit access rights and privileges [15]. Traditional access management systems look up these rights against a central server database.

1.4.4 Security Challenges in IoT

1.4.4.1 Attack Vectors on IoT Devices

The widespread deployment of IoT architectures has expanded the digital attack surface of modern systems [16]. IoT configurations expose physical hardware directly to interconnected layers, rendering them vulnerable to various exploits.

At the physical layer, hardware components are frequently deployed in unattended environments, making them susceptible to the following attack vectors :

- **Node Replication** : An attacker can physically capture a device, extract its credentials or cryptographic keys, and inject a replicated malicious node into the network to falsify transmitted data.
- **Tampering** : Direct physical access allows modification of the device firmware or extraction of locally stored information via unprotected physical interfaces.
- **Side-Channel Attacks** : By observing the physical characteristics of the device such as power consumption or electromagnetic emissions during computations, an attacker can deduce secret keys.

1.4.4.2 Access Control Attacks

Beyond the physical layer, the management of security at the access control level faces major logical threats targeting the legitimacy of access rights. Two main attacks are identified :

- **Collusion Attack** : This threat occurs when two malicious users, neither of whom individually possesses the required privileges to access data, decide to combine their respective attribute keys to bypass the access policy and decrypt the message. Access control systems must structurally prevent the combination of keys from distinct entities from generating a valid authorization.
- **Replay Attack** : An attacker intercepts a valid transmission containing a legitimate identifier, session key, or access right, then retransmits it to the access control mechanism at a later time to fraudulently authenticate without possessing up-to-date keys.

1.5 Access Control Mechanisms

1.5.1 Discretionary Access Control (DAC)

Discretionary Access Control (DAC) is a traditional user-centric access framework where the owner of a data resource exerts complete administrative control over who can view or modify that resource [15]. Access permissions are typically maintained via localized Access Access Control Lists (ACLs). While DAC provides a high degree of flexibility, it suffers from significant security

vulnerabilities, as permissions can be forwarded by authorized users to untrusted entities without the explicit consent or knowledge of the system administrator.

1.5.2 Mandatory Access Control (MAC)

Mandatory Access Control (MAC) represents a rigid, highly secure access framework where access permissions are dictated strictly by a centralized operating authority based on hierarchical clearance tiers [15]. System elements and individual objects are assigned clear security classifications (e.g., public, confidential, secret), and users can only access data if their personal clearance level matches or exceeds the object’s classification. This centralized enforcement structure lacks the elasticity needed for dynamic organizational setups.

1.5.3 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) simplifies access provisioning by grouping individual permissions into distinct, formalized roles (e.g., Engineer, Manager, Technician) rather than assigning access directly to individual accounts [12]. Users are then mapped to one or more roles, granting them the associated permissions. While RBAC lowers administrative overhead in standard networks, it struggles with scalability when applied to decentralized IoT spaces that require dynamic access adjustments based on context.

1.5.4 Attribute-Based Access Control (ABAC)

Attribute-Based Access Control (ABAC) addresses the limitations of RBAC by evaluating access decisions using fine-grained characteristics called attributes [15]. These attributes are divided into four main categories : user profiles (e.g., department, job status), resource metadata (e.g., file classification, device source), environmental context (e.g., request time, geographic location), and action targets (e.g., read, write). ABAC uses a central policy engine to evaluate policy statements, enabling highly precise access control decisions.

1.5.5 Limitations of Existing Mechanisms

While the access control mechanisms presented above each offer certain advantages, they all share a fundamental limitation when applied to distributed IoT environments. Traditional implementations of DAC, MAC, RBAC, and ABAC often rely on centralized management entities responsible for defining and enforcing access policies. In a setting where multiple independent organizations coexist, this centralization becomes a bottleneck and a single point of failure. Furthermore, traditional mechanisms such as DAC, MAC, and RBAC do not provide cryptographic enforcement of access control. This means that access decisions are made at the application level and can potentially be bypassed if the underlying system or database is compromised.

ABAC addresses some of these limitations by introducing attribute-based policies, but it still depends on a single central authority to manage all attributes and enforce policies. These

limitations motivate the use of a cryptographic access control mechanism that is decentralized, attribute-based, and mathematically enforceable, leading to the use of MA-ABE.

1.6 Attribute-Based Encryption

1.6.1 Identity-Based Encryption (IBE)

IBE is an asymmetric cryptographic paradigm where a user’s public key can be derived from an arbitrary, unique string identifier, such as an email address [14]. This eliminates the need for a complex Public Key Infrastructure (PKI) and digital certificates. A central Master Private Key generator creates the matching secret decryption keys. However, IBE is designed for one-to-one communications and cannot natively handle one-to-many communication or policy-driven data distribution rules.

1.6.2 Attribute-Based Encryption (ABE)

Attribute-Based Encryption (ABE) generalizes IBE by embedding access control logic directly into the cryptographic operations [14]. Instead of targeting a specific user identifier, ciphertexts are bound to logical access rules, and user decryption keys are tied to descriptive attributes. Decryption is only mathematically possible if the attribute set associated with a user’s private key satisfies the logical conditions built into the ciphertext.

1.6.3 MA-ABE

Standard ABE systems rely on a single central authority to manage all system attributes and issue every private key. This central authority represents a severe security vulnerability and a single point of failure. MA-ABE solves this limitation by splitting attribute management across multiple independent entities [10]. Each authority can independently manage its own sub-universe of attributes and issue decryption keys to users without centralized coordination. To prevent users from pooling different keys together to bypass access rules, MA-ABE links keys to a unique global identifier (GID), preventing collusion attacks.

1.6.4 Why MA-ABE for IoT Security

In distributed IoT environments, the primary challenge is to enforce access control over data in a decentralized manner, where multiple policies can coexist independently. MA-ABE addresses this challenge for several reasons. First, MA-ABE allows the definition of flexible access policies based on user attributes, providing fine-grained and cryptographically enforced access control. Only users whose attributes satisfy the specified access policy can decrypt the protected data. Second, MA-ABE supports multiple independent authorities, each managing their own set of attributes. This decentralized architecture is naturally suited to environments

where multiple departments or organizations coexist. Third, MA-ABE provides resistance against collusion attacks, ensuring that multiple users cannot combine their attribute keys to decrypt a ciphertext that neither of them could decrypt alone. Finally, because MA-ABE is computationally expensive for encrypting large volumes of data, it is commonly integrated into a hybrid encryption scheme.

1.7 AES Symmetric Encryption

1.7.1 What is AES ?

The AES is a symmetric-key block cipher standardized by the National Institute of Standards and Technology (NIST) in 2001 as the global standard for secure data processing [17]. AES operates on a fixed block dimension of 128 bits (16 bytes) and supports variable cryptographic key lengths of 128, 192, or 256 bits. The algorithm employs a series of structured mathematical substitutions, permutations, and linear transformations executed over multiple internal rounds.

1.7.2 AES-CBC Mode

Cipher Block Chaining (CBC) is a highly reliable mode of operation for block ciphers that introduces sequential dependencies among encrypted data blocks [17]. In CBC mode, each plaintext block is combined with the immediately preceding ciphertext block using a bitwise XOR operation before being passed into the encryption engine. To ensure that encrypting identical plaintext records yields completely distinct ciphertext segments, the very first data block is XORed with a unique, pseudorandom bit array known as an Initialization Vector (IV). This dependency prevents pattern disclosure and structural analysis vulnerabilities across multi-block messages.

1.7.3 Why AES is Suitable for IoT ?

ES is particularly well suited for IoT environments for several reasons. First, as a symmetric encryption algorithm, it is computationally lightweight compared to asymmetric encryption, which is critical in resource-constrained environments where processing power and memory are limited [5]. Second, AES operates on fixed-size blocks of 16 bytes, making it highly efficient for encrypting short messages such as sensor readings. Finally, AES-128 provides a sufficient level of security for IoT applications while keeping the computational cost minimal, making it a standard choice for data protection at the edge layer.

1.8 Blockchain Fundamentals

1.8.1 What is Blockchain ?

A blockchain is a decentralized, distributed ledger system that records transactions across an interconnected network of computers in an unalterable and chronological sequence [18]. Each block in the chain contains a cryptographic hash of the previous block, a timestamped batch of validated transactions, and a cryptographic puzzle solution. This design ensures that modifying any historical data block requires recalculating all subsequent blocks and gaining consensus across a majority of the network nodes, making the ledger highly resistant to retroactive data modification.

1.8.2 Smart Contracts

Smart Contracts are self-executing software programs deployed directly onto a blockchain network [15]. They automatically execute predefined programmatic instructions when specific, verifiable conditions are met. Because smart contracts run on decentralized consensus infrastructure, they eliminate the need for an intermediate third-party broker or central coordinator. In IoT security platforms, smart contracts can manage the registration of cryptographic parameters and enforce unalterable tracking of access configurations.

1.8.3 Ethereum and Ganache

Ethereum expanded blockchain technology by introducing a built-in Turing-complete programming environment called the Ethereum Virtual Machine (EVM), which executes smart contracts globally [18]. To build and evaluate decentralized software without incurring actual financial fees or transactional latencies, developers use local simulation networks. Ganache provides a localized, deterministic Ethereum blockchain environment that instantiates virtual testing accounts, processes transactions instantly, and supports full execution inspection, making it a widely used platform for development and testing.

1.8.4 Blockchain for Secure Data Management

In decentralized environments, blockchain technology provides a shared and tamper-resistant ledger that can be accessed and verified by multiple participants [12, 18]. Its decentralized nature eliminates the need for a central controlling entity, while its immutability ensures that recorded information cannot be altered without network consensus. These properties make blockchain a suitable solution for applications requiring transparency, traceability, integrity, and distributed trust.

1.9 Conclusion

In this chapter, we reviewed the fundamental concepts of IoT security, analyzed the main access control mechanisms, and identified their limitations in decentralized environments. We introduced the architectural roles of Fog and Cloud computing, and highlighted how decentralized blockchain technology helps ensure data integrity. Finally, we presented two important cryptographic approaches for IoT security : symmetric encryption for efficient data protection and attribute-based encryption for enforcing access control.

The limitations of traditional access control mechanisms and the growing need for decentralized security solutions motivate the adoption of cryptographically enforced access control and distributed trust models in IoT environments. The next chapter presents the architecture of the proposed system and details the design choices made to address these challenges.

CHAPITRE 2

SYSTEM DESIGN AND ARCHITECTURE

2.1 Introduction

Following the comprehensive review of the cryptographic mechanisms and security challenges presented in Chapter 1, this chapter delineates the actual design and technical architecture of the proposed secure data-sharing system.

The primary objective of this architecture is to enforce a secure, attribute-based, and decentralized access control layout tailored for resource-constrained Internet of Things (IoT) components. This design explicitly minimizes the processing overhead on edge hardware by delegating computationally intensive operations to a Fog node, while anchoring trust information in an immutable decentralized ledger.

2.2 Objectives of the system designed

The proposed system is designed to ensure data security and efficient operation in resource-constrained IoT environments. Its main objectives are :

- Ensure secure access control by using Multi-Authority Attribute-Based Encryption (MA-ABE), making data access impossible for unauthorized entities.
- Offload computationally intensive cryptographic operations from resource-constrained IoT devices to fog nodes, which possess greater processing and storage capabilities.
- Utilize decentralized blockchain technology to provide secure storage and traceability of MA-ABE-encrypted keys, thereby removing dependence on centralized servers.
- Leverage a cloud infrastructure for storing encrypted data, while keeping cryptographic keys and sensitive parameters outside its control to preserve data confidentiality.

2.3 System Actors and Entities

The proposed system relies on seven distinct operational entities, each responsible for a specific role, as described below :

2.3.1 IoT Node

The IoT node represents an edge sensing device responsible for periodically collecting measurements and generating data, which are then transmitted to higher system layers for processing and storage. Given its limited resources, it is restricted to lightweight cryptographic operations, particularly symmetric encryption using AES, before data transmission.

2.3.2 Data Owner

The Data Owner is the entity responsible for the data generated by the IoT node. It defines the access policies under which the keys must be protected and ensures their transmission to subsequent processing entities.

2.3.3 Attribute Authority

The Attribute Authority is responsible for managing system attributes and issuing cryptographic keys to authorized entities. It generates the public parameters required by the encryption scheme and maintains the authority secrets necessary for key generation.

2.3.4 Fog Node

The Fog node acts as an intermediate processing layer between edge devices and the cloud infrastructure. It handles computationally intensive operations, including MA-ABE encryption, to ensure the protection of encryption keys, and coordinates interactions with the blockchain as well as cloud storage.

2.3.5 Blockchain Ledger

The blockchain provides decentralized and immutable storage for access-key-related information. It guarantees data integrity and prevents unauthorized modification, tampering, or deletion of registered records.

2.3.6 Cloud Server

The cloud server provides long-term storage capacity for collected data and ensures their availability for future retrieval. It stores encrypted data without possessing any cryptographic keys and therefore cannot decrypt the stored data.

2.3.7 Data User

The Data User represents an entity requesting access to the data managed by the system. It interacts with the different components to obtain the required information and can only access it if its attributes satisfy the defined access policies.

2.4 System diagrams

This section presents the Unified Modeling Language (UML) diagrams that model the structure and behavior of the proposed system.

2.4.1 Use Case Diagram

The use case diagram shown in Figure 2.1 illustrates the external entities (actors) that interact with the proposed system, as well as the functionalities it provides. Each use case is subsequently described through a step-by-step scenario.

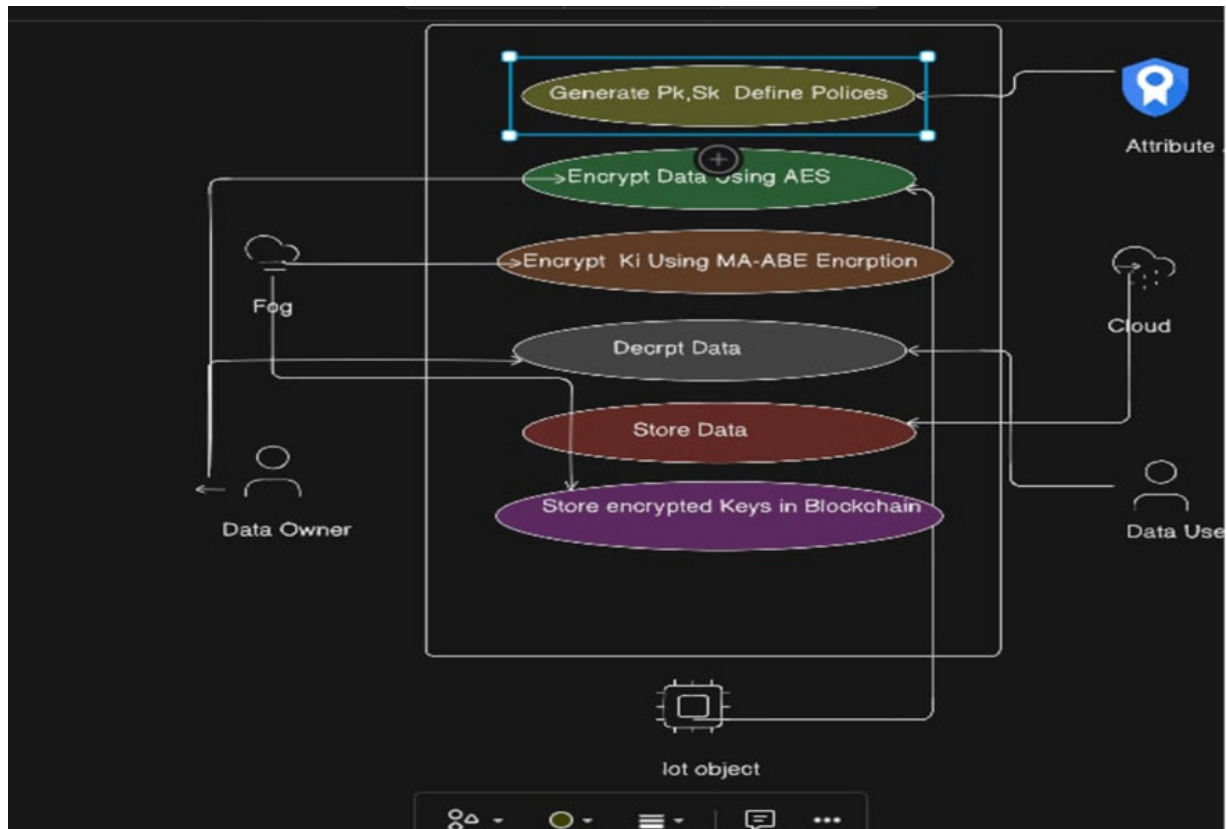


FIG. 2.1 : Use case diagram

2.4.2 Class diagram

The class diagram in Figure 2.1 provides a structural view of the proposed system by illustrating the main entities involved and the relationships between them

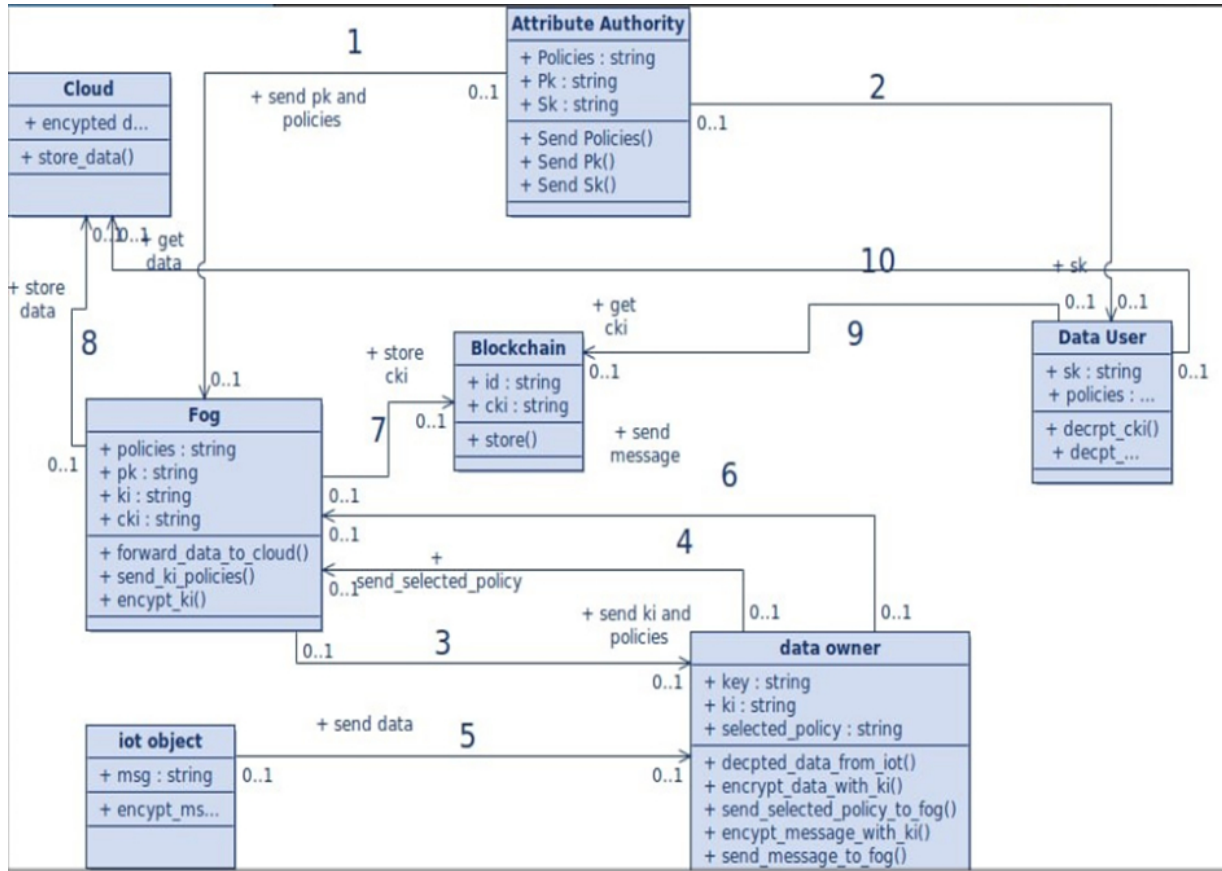


FIG. 2.2 : Class diagram

The system is composed of seven main classes :

- **Attribute Authority** : Responsible for managing attributes, defining access policies, and generating the cryptographic credentials required by the system entities.
- **IoT Node** : Represents the sensing device that collects data from the physical environment and transmits it to the Data Owner.
- **Data Owner** : Manages the data generated by the IoT node, defines the applicable access policy, and coordinates the protection of the collected data.
- **Fog Node** : Acts as an intermediate processing layer responsible for performing computationally intensive operations and coordinating interactions between the different system components.
- **Blockchain Ledger** : Provides decentralized and immutable storage for cryptographic records required during data access.

- **Cloud Server** : Stores encrypted data and ensures their availability for future retrieval.
- **Data User** : Requests access to the protected data and retrieves information when its attributes satisfy the corresponding access policies.

2.5 Process workflow and protocols

The core operations of the system are executed across three separate operational phases : system initialization, data collection and storage, and data decryption and access.

2.5.1 Phase 1 : System initialization

This phase follows a deterministic process :

1. The Attribute Authority initializes the cryptographic environment by generating the global parameters, the required public and private keys, and the set of attributes. The Fog node obtains the public keys (PK), while the Data User receives the private keys (SK) corresponding to its assigned attributes.
2. The Data Owner sends a request to the Fog node to obtain a unique identifier (ID).
3. The Fog node generates a ID and a symmetric encryption key (K_i) associated with the Data Owner. It then transmits the ID, the key K_i , and the list of available access policies to the Data Owner.
4. The Data Owner receives the key K_i and later uses it to encrypt data during the next phase. It then selects the access policy (P) to be applied to the generated data and sends it to the Fog node.
5. The Fog node encrypts the symmetric encryption key (K_i) using the MA-ABE scheme according to the access policy P defined by the Data Owner, producing the encrypted key CK_i . Through a smart contract, it then records the Data Owner's identifier (ID) and the corresponding encrypted key CK_i on the blockchain.

2.5.2 Phase 2 : Data collection and storage

During this phase, the following operations are performed :

- IoT nodes generate sensor data (M_j) at the network edge. These data are then encrypted using the AES-128 algorithm, producing the ciphertext (CT), and subsequently transmitted to the Data Owner.
- The Data Owner receives the encrypted data (CT) and decrypts them to recover the original data (M_j). The data are then re-encrypted using AES-128 with its own symmetric key (K_i), producing the ciphertext (CT). The Data Owner subsequently sends its identifier (ID) together with (CT) to the Fog node.

- Upon receiving (ID) and (CT) , the Fog node first verifies that the Data Owner has completed the registration phase by consulting the information recorded on the blockchain. If the identifier (ID) is found, the Fog node stores the ciphertext (CT) in the cloud in a structured manner, associating it with the corresponding ID .

2.5.3 Phase 3 : Data decryption and Access

Data decryption and access are performed according to the following steps :

1. The Data User retrieves the encrypted key (CK_i) from the blockchain and requests the encrypted data (CT) associated with the targeted Data Owner from the cloud server. The cloud server then returns the stored ciphertext.
2. The Data User possesses a secret key (SK) , issued by the Attribute Authorities according to its assigned attributes. It uses this key to decrypt the encrypted key (CK_i) and recover the symmetric key (K_i) . This operation is only possible if the user's attributes satisfy the access policy associated with (CK_i) .
3. Once the key (K_i) is obtained, the Data User decrypts the ciphertext (CT) retrieved from the cloud using the AES-128 algorithm and the key (K_i) , thereby recovering the original sensor data (M_j) .

2.6 Smart contract design

The blockchain component of the system is implemented through a smart contract that acts as a decentralized registry for storing the cryptographic records required during data access. The smart contract provides three main functionalities :

- Registration of cryptographic records (ID, CK_i) associated with a Data Owner.
- Retrieval of previously registered records.
- Verification of the existence of a record associated with a given identifier.

By automating these operations on the blockchain, the smart contract eliminates reliance on a centralized authority. It ensures the integrity of stored records and provides resistance against unauthorized modification or tampering.

2.7 Access control policy design

The system supports three predefined access policies managed by the Attribute Authority $AUTH_1$. Each policy is a boolean expression over attributes that is embedded directly into the MA-ABE ciphertext at encryption time. Table 2.1 provides illustrative examples of the access control policies defined in the system.

TAB. 2.1 : Access control policies defined in the system.

Policy	Policy Expression	Authorized Users
$Policy_1$	$STUDENT@AUTH_1$ OR $STAFF@AUTH_1$	Any user with STUDENT or STAFF attribute
$Policy_2$	$MANAGER@AUTH_1$ AND $RESEARCHER@AUTH_1$	Users holding both MANAGER and RESEARCHER attributes
Admin_Only	$ADMIN@AUTH_1$	Exclusively users holding the ADMIN attribute

2.8 Conclusion

This chapter has provided a detailed architectural map of the proposed system, demonstrating how an IoT node, Fog architecture, an Ethereum-based blockchain ecosystem, and a stateless cloud repository work together to form a complete secure data-sharing platform.

By implementing a hybrid encryption layout where telemetry data is protected via AES-128 while session keys are bound within a Multi-Authority Attribute-Based Encryption scheme and anchored on an immutable blockchain ledger, the architecture effectively maintains data confidentiality without overloading low-power edge hardware.

The following chapter will detail the concrete implementation environment, software libraries, and configurations of this architecture.

CHAPITRE 3

DEVELOPMENT TOOLS AND ENVIRONMENT

3.1 Introduction

This chapter presents all the tools, technologies, and environments used to develop and test the proposed secure IoT data sharing system. The implementation relies on a combination of simulation tools, cryptographic libraries, blockchain frameworks, and Python components. Each tool was chosen based on its suitability for the specific requirements of the system, including resource constraints, cryptographic capabilities, and ease of integration.

3.2 Hardware and software environment

3.2.1 Development machine

The system was developed and tested on a single machine. The specifications of the development machine are as follows :

- Operating System : Kali Linux (x64).
- Processor : Intel Core i5/i7.
- RAM : 8 GB.
- Storage : 256 GB SSD.
- Python Version : Python 3.10.

All components of the system, including the blockchain, the simulation, and all Python servers, were run simultaneously on this single machine during testing.

3.2.2 Python environment

Python 3.10 was used as the main programming language for all system components except the IoT node firmware. All Python dependencies were managed using a virtual environment called `iot_env`, which isolates the project dependencies from the system Python installation.

The following components were implemented in Python :

- Attribute Authority.
- Fog Node.
- Data Owner.
- Cloud Server.
- Data User.

3.3 IoT simulation environment

3.3.1 Contiki-NG

Contiki-NG [6] is an open-source operating system specifically designed for resource-constrained IoT devices. It provides a lightweight multitasking environment that supports low-power wireless communication protocols and is widely used in academic and industrial IoT research.

In our system, Contiki-NG was used to implement the IoT node firmware. The node was written in C and compiled using the Contiki-NG toolchain. It periodically generates temperature readings, encrypts them using a pre-shared AES key, and transmits them through a serial connection.

Contiki-NG was chosen for the following reasons :

- It is specifically designed for constrained IoT devices with limited memory and processing power.
- It provides a well-documented development environment for sensor node programming.
- It integrates natively with the Cooja network simulator.

3.3.2 Cooja Network Simulator

Cooja [19] is the official network simulator of Contiki-NG. It allows developers to simulate IoT networks without requiring physical hardware. Each simulated device, called a mote, runs the actual compiled Contiki-NG firmware, making the simulation as close to real hardware behavior as possible.

In our system, Cooja was used to simulate the IoT temperature sensor node. The node runs inside Cooja and communicates with the Data Owner Python server through a Serial Socket connection on port 60001.

Cooja was chosen because :

- It allows testing IoT firmware without physical devices.
- It supports Serial Socket communication which enables integration with external Python components.
- It is free, open-source, and well integrated with Contiki-NG.

3.4 Blockchain Environment

3.4.1 Ganache

Ganache is a local Ethereum blockchain simulator developed by Truffle Suite. It provides a personal blockchain environment for development and testing purposes. Ganache runs locally on the development machine and provides a set of pre-funded Ethereum accounts that can be used immediately without any configuration.

In our system, Ganache was used as the blockchain backend. The Fog node interacts with Ganache to store the MA-ABE encrypted ciphertext CKi on the blockchain, and the Data User retrieves it from there.

Ganache was chosen because :

- It provides a fully functional local Ethereum blockchain with no setup complexity.
- It is free and runs entirely on the local machine with no internet connection required.
- It is compatible with Web3.py and Solidity smart contracts.

3.4.2 Solidity and smart contract

Solidity is a high-level programming language designed for writing smart contracts on the Ethereum blockchain. Smart contracts are self-executing programs stored on the blockchain that run automatically when predefined conditions are met.

In our system, a smart contract called UserRegistry was written in Solidity. It exposes the following functions :

- registerUser - stores a user's CKi on the blockchain.
- getUser CK_i - retrieves a stored CKi by user ID.
- isUserRegistered - checks whether a user already has a stored CK_i .

The smart contract guarantees that once a CKi is stored, it cannot be modified or deleted, ensuring the integrity of the cryptographic material.

3.4.3 py-solc-x

py-solc-x [20] is a Python library that provides a wrapper around the Solidity compiler (solc). It allows Solidity smart contracts to be compiled directly from Python code without requiring a separate compilation step.

In our system, py-solc-x was used to compile the UserRegistry smart contract and deploy it to Ganache at the start of each session. The resulting contract address was then automatically updated in all relevant Python files.

3.4.4 Web3.py

Web3.py is a Python library for interacting with the Ethereum blockchain. It provides functions for connecting to a blockchain node, deploying smart contracts, and calling contract functions.

In our system, Web3.py was used in two components :

- The Fog node uses Web3.py to call the RegisterUser function and store the CKi on the blockchain after MA-ABE encryption.
- The Data User uses Web3.py to call the GetUserCKi function and retrieve the CKi from the blockchain.

Web3.py was chosen because :

- It is the standard Python library for Ethereum blockchain interaction.
- It is fully compatible with Ganache.
- It provides a simple and clean API for smart contract interaction.

3.5 Cryptographic environment

3.5.1 Charm-Crypto Library

Charm-Crypto is an open-source Python framework for rapid prototyping of cryptographic schemes. It provides implementations of advanced cryptographic primitives including pairing-based cryptography, which is the mathematical foundation of MA-ABE.

In our system, Charm-Crypto was used to implement the entire MA-ABE scheme. Specifically :

- The MaabeRW15 scheme was used for multi-authority attribute-based encryption.
- The PairingGroup class with the SS512 curve was used for bilinear pairing operations.

- The `objectToBytes` and `bytesToObject` functions were used for serializing and deserializing MA-ABE ciphertexts for blockchain storage.

Charm-Crypto was chosen because :

- It provides a ready-to-use implementation of MA-ABE that would otherwise require hundreds of lines of low-level mathematical code.
- It is well documented and widely used in academic cryptography research.
- It integrates naturally with Python.

3.5.2 PyCryptodome

PyCryptodome is a Python library that provides implementations of standard cryptographic algorithms including AES, RSA, and SHA. It is a self-contained library with no external dependencies.

In our system, PyCryptodome was used to implement AES-128 in CBC mode for data encryption and decryption. Specifically :

- The IoT node encrypts temperature data using a pre-shared AES key.
- The Data Owner decrypts the IoT data and re-encrypts it using the session AES key derived from `ki_hex`.
- The Data User uses the recovered AES key to decrypt the messages stored in the Cloud.

PyCryptodome was chosen because :

- It provides a simple and well-documented AES implementation.
- It supports CBC mode with PKCS7 padding out of the box.
- It is lightweight and easy to integrate.

3.6 System communication

3.6.1 Serial Socket - Cooja to Python

The Serial Socket is a feature provided by Cooja that allows a simulated mote to communicate with an external process through a TCP socket. In our system, the Serial Socket was configured on port 60001 and used to transmit encrypted temperature data from the Cooja IoT node to the Data Owner Python server.

This communication channel replaces the physical serial port that would be used in a real hardware deployment, making it possible to test the full system without any physical IoT devices.

3.6.2 TCP sockets - Between Python components

All communication between the Python components of the system was implemented using standard TCP sockets. The following connections exist in the system :

- Data Owner → Fog on port 5000 — sends re-encrypted IoT data.
- Fog → Cloud on port 6000 — forwards encrypted messages for storage.
- Data User → Cloud on port 8001 — requests stored encrypted messages.

TCP was chosen for its reliability and simplicity. Since all components run on the same machine during testing, network latency is not a concern.

3.7 Conclusion

This chapter presented all the tools and technologies used to implement the proposed system. The combination of Contiki-NG and Cooja provides a realistic IoT simulation environment, while Ganache and Solidity provide a functional blockchain backend. The Python ecosystem, particularly Charm-Crypto, Web3.py, and PyCryptodome, provides all the cryptographic and networking capabilities required by the system. Together these tools form a complete and coherent development environment that allowed the full implementation and testing of the proposed secure IoT data sharing system.

CHAPITRE 4

IMPLEMENTATION AND TESTING

4.1 Introduction

This chapter presents the practical implementation of the proposed secure IoT data sharing system. The system was developed and tested in a controlled environment using a combination of simulation tools, Python scripts, and a local blockchain.

This chapter successively presents the implementation of each component, the scenario-based tests ensuring validation of the system's correctness and security, and finally the problems encountered and their solutions.

The implementation of our secure platform is detailed through a scenario comprising IoT nodes and Fog nodes, which serves to illustrate the decentralized nature of the proposed access control system, along with the blockchain-based key storage approach.

4.2 Implementation of the system components

The system startup follows a strict order. First, Ganache is started to provide the local blockchain environment, as illustrated in Figure 4.1. Next, the smart contract [21] is compiled and deployed using `py-solc-x` [20], and the resulting contract address is automatically updated in all relevant Python files. Finally, the Fog node, the Data Owner, and the Cloud server are started sequentially.

```

(iot_env)aymen@kali: ~/contiki-ng/examples/iot_project/python
compiled = compile_source(source, output_values=['abi', 'bin'], solc_version='0.8.0')
contract_id, contract_interface = list(compiled.items())[0]
contract = w3.eth.contract(abi=contract_interface['abi'], bytecode=contract_interface['bin'])
tx_hash = contract.constructor().transact({'from': w3.eth.accounts[0]})
receipt = w3.eth.wait_for_transaction_receipt(tx_hash)
addr = receipt.contractAddress
print("CONTRACT ADDRESS:", addr)
import re
for fname in ['fog1.py', 'data_user.py']:
    with open(fname, 'r') as f:
        content = f.read()
        content = re.sub(r'0x[a-fA-F0-9]{40}', addr, content)
    with open(fname, 'w') as f:
        f.write(content)
    print(f"Updated {fname}")
EOF
CONTRACT ADDRESS: 0xeebA1539FFF44F1CEd9400cb40920Fb9c5F125f8
Updated fog1.py
Updated data_user.py

(iot_env)-(aymen@kali)-[~/contiki-ng/examples/iot_project/python]
$

```

FIG. 4.1 : Smart Contract Deployment

4.2.1 Phase 1 : System initialization

The system initialization phase concerns the attribute authority as well as the fog node :

- **Attribute Authority**

The attribute authority generates the Global Public Parameters (GPP), as illustrated in Figure 4.2, as well as the public and master secret keys of the $AUTH_1$ authority. It also generates secret keys for Data Users according to their assigned attributes. In this implementation, the authority manages the following attributes : *STUDENT*, *STAFF*, *MANAGER*, *RESEARCHER*, and *ADMINISTRATOR*.

```

(iot_env)aymen@kali: ~/contiki-ng/examples/iot_project/python

✅ Pairing group created. Order: 730750818665451621361119245571504901405976559617

[2/3] Loading MA-ABE Scheme (YJ14)...
✅ Loaded saved GPP from gpp_keys.json
✅ Global parameters (GPP) generated successfully.

[3/3] Setting up Authority 'AUTH1' and generating Master Keys...
✅ Loaded saved authority keys from authority_keys.json
✅ Authority master setup completed:
🔑 Master Secret Keys (SK): ['alpha', 'beta', 'gamma'] → [STAY HERE - SECURE]
🔑 Master Public Keys (PK): ['e_alpha', 'g_beta', 'g_beta_inv'] → [SAFE TO SHARE]

=====
🚀 SYSTEM FULLY INITIALIZED BY AUTHORITY
=====

■ FOG NODE STARTED

Listening on 127.0.0.1:5000

```

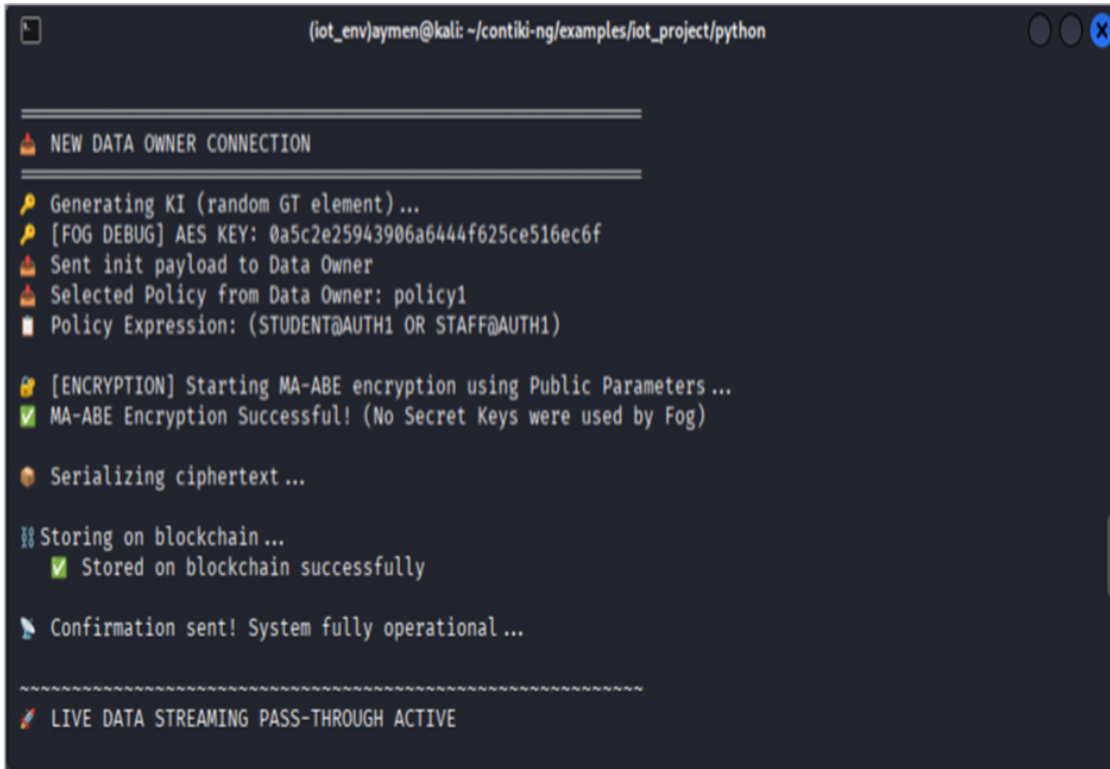
FIG. 4.2 : Attribute Authority Terminal

The authority then distributes the public key to the Fog node and the secret keys to Data Users according to their respective attributes.

- **Fog Node**

The Fog node operates in coordination with the Data Owner and the blockchain, is illustrated in Figure 4.3 and proceeds as follows :

1. The Data Owner sends a request to the Fog node to obtain a ID.
2. The Fog node generates a unique identifier (ID) and a symmetric encryption key (K_i). It then sends the ID , the key K_i , and the list of available access policies to the Data Owner.
3. The Data Owner defines the access policy associated with the data and transmits it to the Fog node.
4. The Fog node encrypts the key K_i using the MA-ABE scheme according to the access policy defined by the Data Owner (e.g., $STUDENT@AUTH_1$ OR $STAFF@AUTH_1$). This operation produces the encrypted key CK_i , which is subsequently stored on the blockchain through the smart contract.



```
(iot_env)aymen@kali: ~/contiki-ng/examples/iot_project/python

=====
🔥 NEW DATA OWNER CONNECTION
=====

🔑 Generating KI (random GT element)...
🔑 [FOG DEBUG] AES KEY: 0a5c2e25943906a6444f625ce516ec6f
🔥 Sent init payload to Data Owner
🔥 Selected Policy from Data Owner: policy1
📋 Policy Expression: (STUDENT@AUTH1 OR STAFF@AUTH1)

🔑 [ENCRYPTION] Starting MA-ABE encryption using Public Parameters ...
✅ MA-ABE Encryption Successful! (No Secret Keys were used by Fog)

🔑 Serializing ciphertext ...

🔑 Storing on blockchain ...
✅ Stored on blockchain successfully

🔑 Confirmation sent! System fully operational ...

=====
🔥 LIVE DATA STREAMING PASS-THROUGH ACTIVE
```

FIG. 4.3 : Fog Node Terminal

The smart contract exposes the ‘registerUser’ function, which enables the registration of the (ID , CK_i) pair associated with a Data Owner, as illustrated in Figure 4.4.

```

(iot_env)aymen@kali: ~
Mnemonic:      athlete ask month margin among dynamic egg wedding inject key either nose
Base HD Path:  m/44'/60'/0'/0/{account_index}

Default Gas Price
=====
2000000000

BlockGas Limit
=====
30000000

Call Gas Limit
=====
50000000

Chain
=====
Hardfork: shanghai
Id:      1337

RPC Listening on 0.0.0.0:8545

```

FIG. 4.4 : Ganache terminal

4.2.2 Phase 2 : Data collection and storage

Data collection and storage include the following elements :

- **IoT Node**

The IoT node was implemented in the C programming language under the Contiki-NG operating system and simulated using the Cooja network simulator [19]. As illustrated in Figure 4.5, the node periodically generates temperature readings, encrypts them using the AES-128 algorithm with a pre-shared key, and then transmits the encrypted data to the Data Owner.

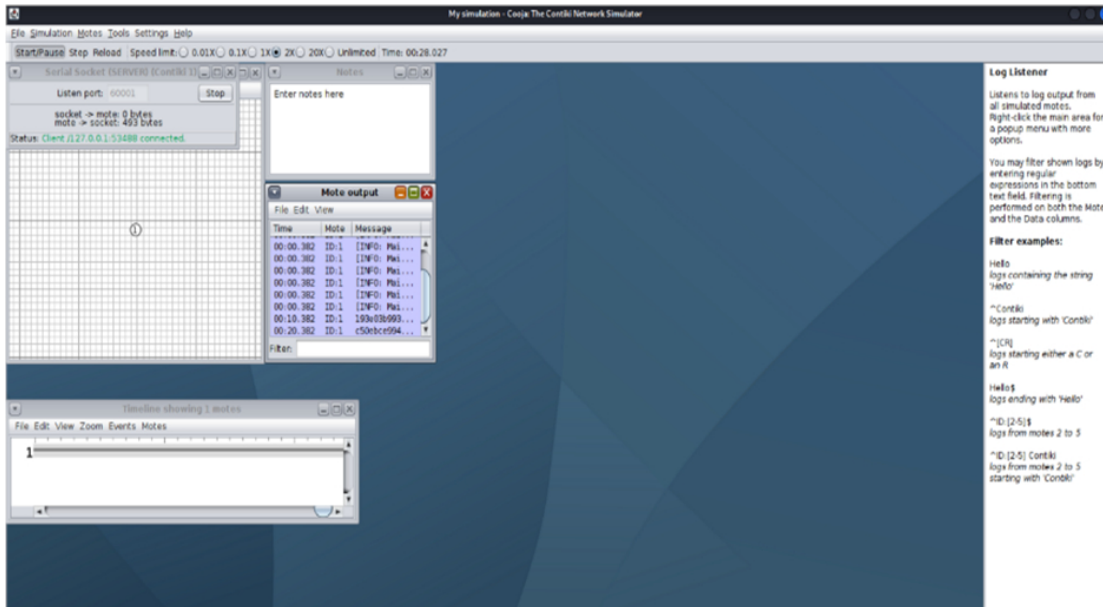
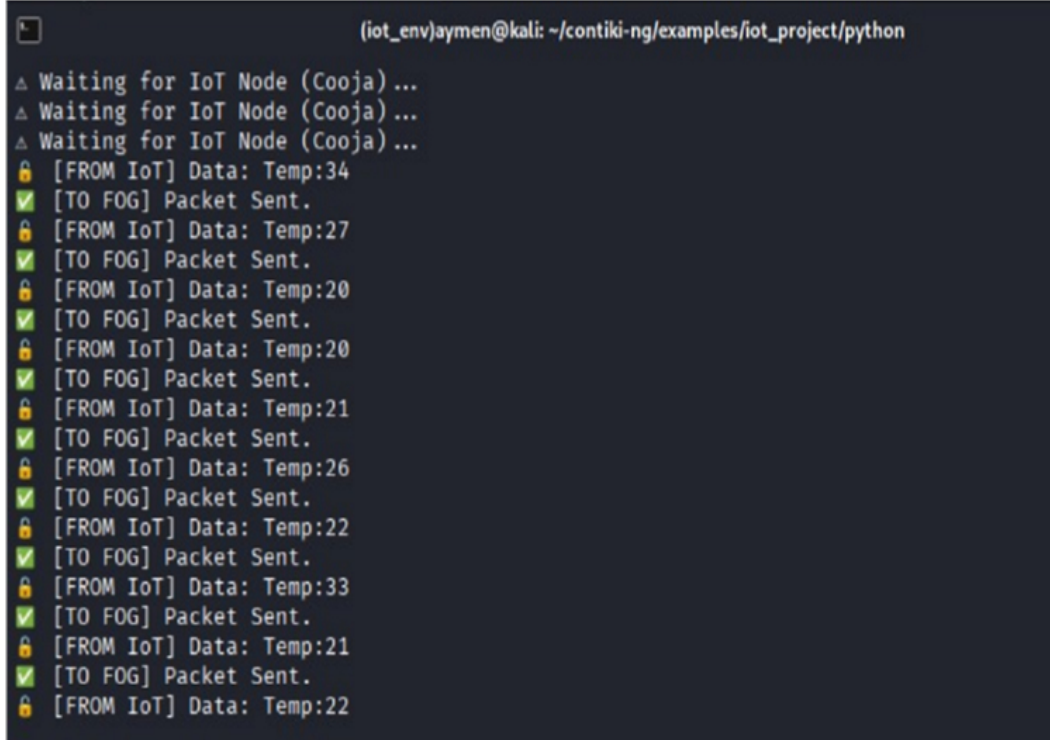


FIG. 4.5 : Cooja network simulator

- **Data Owner**

Upon receiving the encrypted data, the Data Owner decrypts them using the pre-shared key to recover the original measurements. The data are then re-encrypted using the symmetric key K_i received from the Fog node, producing the ciphertext CT . The re-encrypted data are subsequently transmitted to the Fog node, as illustrated in Figure 4.6.



```
(iot_env)aymen@kali: ~/contiki-ng/examples/iot_project/python
△ Waiting for IoT Node (Cooja) ...
△ Waiting for IoT Node (Cooja) ...
△ Waiting for IoT Node (Cooja) ...
🔒 [FROM IoT] Data: Temp:34
✅ [TO FOG] Packet Sent.
🔒 [FROM IoT] Data: Temp:27
✅ [TO FOG] Packet Sent.
🔒 [FROM IoT] Data: Temp:20
✅ [TO FOG] Packet Sent.
🔒 [FROM IoT] Data: Temp:20
✅ [TO FOG] Packet Sent.
🔒 [FROM IoT] Data: Temp:21
✅ [TO FOG] Packet Sent.
🔒 [FROM IoT] Data: Temp:26
✅ [TO FOG] Packet Sent.
🔒 [FROM IoT] Data: Temp:22
✅ [TO FOG] Packet Sent.
🔒 [FROM IoT] Data: Temp:33
✅ [TO FOG] Packet Sent.
🔒 [FROM IoT] Data: Temp:21
✅ [TO FOG] Packet Sent.
🔒 [FROM IoT] Data: Temp:22
```

FIG. 4.6 : Data owner terminal

- **Fog Node**

The Fog node receives the re-encrypted ciphertext CT from the Data Owner. Before forwarding it to the cloud, it verifies that the Data Owner has successfully completed the registration phase by consulting the information stored on the blockchain [22]. If the corresponding identifier is found, the storage process is authorized, and the encrypted data are transmitted to the cloud server.

- **Cloud Server**

The cloud server receives the encrypted data CT sent by the Fog node and stores them in a structured manner, as illustrated in Figure 4.7. Each record contains the unique identifier of the Data Owner ID and the corresponding encrypted data CT .

The cloud server does not possess the symmetric key K_i or any information required to obtain it. Consequently, it is unable to decrypt the stored data.

```
(iot_env)aymen@kali: ~/contiki-ng/examples/iot_project/python
$ python3 cloud.py

CLOUD SERVER STARTING

Port 6000 → receives data from Fog
Port 8001 → answers Data User queries
✓ [STORAGE] Loaded storage.json
👤 Owners loaded : 0
■ Last msg ID : 0
✓ [FOG-LISTENER] Listening on 127.0.0.1:6000
✓ [CLOUD] Both listeners active
✓ [USER-LISTENER] Listening on 127.0.0.1:8001

● [FOG] Connection from 127.0.0.1:53206

FOG MESSAGE - 127.0.0.1:53206

✓ [STORAGE] Saved → storage.json
🔥 [STORED] msg #1
```

FIG. 4.7 : Cloud Server Terminal

4.2.3 Phase 3 : Data Decryption and Access

This phase involves the data user and their interaction with the blockchain and the Cloud :

- **Interaction with the Blockchain**

The Data User retrieves the encrypted key CK_i from the blockchain using the ‘getUserCKi’ function exposed by the smart contract. The user then employs the MA-ABE secret key obtained from the Attribute Authority to attempt the decryption of CK_i .

Decryption is possible only if the user’s attributes satisfy the access policy associated with CK_i . For example, if the policy defined during encryption is ” $STUDENT@AUTH_1$ OR $STAFF@AUTH_1$ ”, a user possessing either the $STUDENT@AUTH_1$ or $STAFF@AUTH_1$ attribute can successfully decrypt CK_i and recover the symmetric key K_i .

- **Interaction with the Cloud**

The Data User sends a request to the cloud server to retrieve the encrypted data CT associated with the targeted Data Owner.

Upon receiving the encrypted data, the Data User uses the key K_i and the AES-128 algorithm to decrypt CT and recover the original temperature readings. The interactions between the Data User and the cloud server, including encrypted data retrieval and data decryption, are illustrated in Figure 4.8.

```
(root@msjaymes@kali: ~) cd /opt/ma-abe/examples/iot_project/python

[✓] Secret key generated for attributes: ['STUDENT']

[BLOCKCHAIN] Fetching CKI...
[✓] CKI retrieved from Blockchain.

[DESERIALIZE] Reconstructing MA-ABE Ciphertext...
[✓] Deserialization successful.

[MA-ABE DECRYPTION] Decrypting CKI...
[✓] MA-ABE Decryption Successful!

Recovered AES-128 Key (Hex): 0a5c2e25943906a6444f625ce516ec6f

[CLOUD] Fetching encrypted messages...
[✓] Retrieved 10 message(s) from Cloud.

DECRYPTED MESSAGES

■ Msg #1 [2026-06-05 14:57:07] → Temp:34
■ Msg #2 [2026-06-05 14:57:12] → Temp:27
■ Msg #3 [2026-06-05 14:57:17] → Temp:20
■ Msg #4 [2026-06-05 14:57:22] → Temp:20
■ Msg #5 [2026-06-05 14:57:27] → Temp:21
■ Msg #6 [2026-06-05 14:57:32] → Temp:26
■ Msg #7 [2026-06-05 14:57:37] → Temp:22
■ Msg #8 [2026-06-05 14:57:42] → Temp:33
■ Msg #9 [2026-06-05 14:57:47] → Temp:21
■ Msg #10 [2026-06-05 14:57:52] → Temp:22

[✓] Done! Decrypted 10 messages successfully.
```

FIG. 4.8 : Data User Terminal

4.3 Scenario-Based Testing

This section provides scenario-based test cases that validate the correct functioning of the proposed access control protocol.

4.3.1 Authorized User

In this scenario, the Data User holds the *STUDENT* attribute, which satisfies *policy*₁ : (*STUDENT@AUTH*₁ OR *STAFF@AUTH*₁). The MA-ABE decryption succeeds, the AES session key is recovered, and all encrypted messages are successfully decrypted to reveal the original temperature readings.

4.3.2 Full End-to-End Test

This scenario validates the complete pipeline from IoT data generation to final decryption. The Cooja simulation runs for approximately 20 seconds, generating several temperature readings. All components run simultaneously and communicate correctly. The Data User successfully decrypts all 10 temperature messages collected during the simulation, confirming the end-to-end integrity of the system.

4.4 Encountered Problems and Solutions

This section outlines the issues encountered in such implementations and the solutions we propose.

4.4.1 Blockchain Contract Redeployment

Each time Ganache is restarted, all blockchain data are lost including the deployed contract. This required automating the contract redeployment process and updating the contract address in all Python files at the start of each session. This was solved by writing a deployment script that compiles the contract, deploys it, and automatically updates all relevant files using regular expressions.

4.4.2 Cooja Serial Socket Connection

During testing, the Cooja simulation sometimes failed to connect to the Data Owner through the Serial Socket. This was caused by starting the Data Owner server after launching the simulation. The solution was to always start the Data Owner server before starting the Cooja simulation, ensuring the socket is ready to accept the connection.

4.5 Conclusion

This chapter presented the complete implementation and testing of the proposed secure IoT data sharing system. Each component was implemented and tested individually, and the full end-to-end pipeline was validated through scenario-based testing. The authorized user scenario confirmed that only users with valid attributes can access the IoT data, demonstrating that the access control mechanism correctly enforces the defined policies. The system successfully demonstrates the viability of combining MA-ABE with AES and blockchain for secure, attribute-based IoT data access control.

Conclusion

The rapid proliferation of the IoT has revolutionized modern production environments, but it has simultaneously exposed data systems to significant security and privacy vulnerabilities. Traditional, centralized access control frameworks have proven inadequate for distributed environments, often introducing single points of failure and failing to provide mathematically enforceable security boundaries. This final-year thesis has successfully designed, implemented, and validated a decentralized, cryptographically enforced data-sharing platform tailored for resource-constrained IoT architectures.

By leveraging a hybrid encryption layout, the proposed system elegantly circumvents the hardware bottlenecks inherent to edge infrastructure. Lightweight, high-speed symmetric encryption (AES-128) is deployed at the edge layer to protect raw telemetry. Meanwhile, computationally heavy cryptographic routines are offloaded to an intermediate, resource-rich Fog computing gateway. This gateway secures the data-session keys via MA-ABE, implementing the robust MaabeYJ14 scheme.

Crucially, the integration of an Ethereum-based blockchain (simulated via Ganache) eliminates the reliance on a centralized authority. The deployment of a customized Solidity smart contract (UserRegistry) establishes an unalterable, tamper-proof key registry. The architecture maintains a strict zero-trust storage topology : a public Cloud server functions purely as an encrypted data vault, entirely blinded to the cryptographic keys, while the blockchain separately handles access metadata. The system separates the access validation phase (MA-ABE decryption) from the data recovery phase, guaranteeing that a Data User can only retrieve and decrypt telemetry metrics if their verified attributes mathematically satisfy the on-chain access policies.

Experimental testing executed across the Contiki-NG/Cooja simulation environment, Python frameworks, and Web3.py client interfaces has fully confirmed the real-world viability of this system. Scenario-based evaluations validated that authorized entities (such as users holding the STUDENT attribute matching policy1) can seamlessly perform end-to-end data recovery, while unauthorized actors are cryptographically barred. Ultimately, this work demonstrates that the

combination of MA-ABE, blockchain ledger technology, and Fog computing offers a stateless, scalable, and mathematically bulletproof access control paradigm capable of safeguarding the future of IoT data exchange.

- [1] Luigi ATZORI, Antonio IERA et Giacomo MORABITO. “The Internet of Things : A survey”. In : *Computer Networks* 54.15 (2010), p. 2787-2805. DOI : 10.1016/j.comnet.2010.05.010.
- [2] Ala AL-FUQAHA et al. “Internet of Things : A Survey on Enabling Technologies, Protocols, and Applications”. In : *IEEE Communications Surveys & Tutorials* 17.4 (2015), p. 2347-2376. DOI : 10.1109/COMST.2015.2444095.
- [3] Arindam GIRI et al. “Internet of things (IoT) : a survey on architecture, enabling technologies, applications and challenges”. In : *Proceedings of the 1st International Conference on Internet of Things and Machine Learning. IML '17*. ACM. Liverpool, United Kingdom, 2017. DOI : 10.1145/3109761.3109768.
- [4] ITU-T MEMBERS. *Overview of the Internet of things*. Rapp. tech. Next Generation Networks – Frameworks and functional architecture models. TELECOMMUNICATION STANDARDIZATION SECTOR OF ITU, 2012.
- [5] Jorge GRANJAL, Edmundo MONTEIRO et Jorge SÁ SILVA. “Security for the Internet of Things : A Survey of Existing Protocols and Open Research Issues”. In : *IEEE Communications Surveys & Tutorials* 17.3 (2015), p. 1294-1312. DOI : 10.1109/COMST.2015.2388550.
- [6] George OIKONOMOU et al. “The Contiki-NG open source operating system for next generation IoT devices”. In : *SoftwareX* 18 (2022), p. 101089. DOI : 10.1016/j.softx.2022.101089.
- [7] Sanaz Rahimi MOOSAVI et al. “End-to-end security scheme for mobility enabled healthcare Internet of Things”. In : *Future Generation Computer Systems* 64 (2016), p. 108-124. DOI : 10.1016/j.future.2016.02.020.

- [8] Ankit SHARMA et Kriti BHUSHAN. “A comprehensive survey on IoT security : Challenges, security issues, and countermeasures”. In : *Computer Science Review* 59 (2026), p. 100839. DOI : 10.1016/j.cosrev.2025.100839.
- [9] Rwan MAHMOUD et al. “Internet of things (IoT) security : Current status, challenges and prospective measures”. In : *Proceedings of the 10th International Conference for Internet Technology and Secured Transactions (ICITST)*. IEEE. London, UK, 2015, p. 336-341. DOI : 10.1109/ICITST.2015.7412116.
- [10] Rodrigo ROMAN, Javier LOPEZ et Masahiro MAMBO. “Mobile edge computing, Fog et al. : A survey and analysis of security threats and challenges”. In : *Future Generation Computer Systems* 78 (2018), p. 680-698. DOI : 10.1016/j.future.2016.11.009.
- [11] Manuel DÍAZ, Cristian MARTÍN et Bartolomé RUBIO. “State-of-the-art, challenges, and open issues in the integration of Internet of things and cloud computing”. In : *Journal of Networks and Computer Applications* 67.C (2016), p. 99–117. DOI : 10.1016/j.jnca.2016.01.010.
- [12] Flavio BONOMI et al. “Fog computing and its role in the internet of things”. In : *Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing. MCC '12*. ACM. Helsinki, Finland : Association for Computing Machinery, 2012, p. 13–16. DOI : 10.1145/2342509.2342513.
- [13] Mithun MUKHERJEE et al. “Security and Privacy in Fog Computing : Challenges”. In : *IEEE Access* 5 (2017), p. 19293-19304. DOI : 10.1109/ACCESS.2017.2749422.
- [14] William STALLINGS. *Cryptography and Network Security : Principles and Practice*. NJ, United States : Prentice Hall Press One Lake Street Upper Saddle River, 2010.
- [15] E. BARKA et R. SANDHU. “Framework for role-based delegation models”. In : *Proceedings 16th Annual Computer Security Applications Conference (ACSAC'00)*. IEEE. New Orleans, LA, USA, 2000, p. 168-176. DOI : 10.1109/ACSAC.2000.898870.
- [16] Jie LIN et al. “A Survey on Internet of Things : Architecture, Enabling Technologies, Security and Privacy, and Applications”. In : *IEEE Internet of Things Journal* 4.5 (2017), p. 1125-1142. DOI : 10.1109/JIOT.2017.2683200.
- [17] NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY (NIST). *Advanced Encryption Standard (AES)*. Rapp. tech. Information Technology Laboratory National Institute of Standards et Technology Gaithersburg, MD 20899-8900, 2001.
- [18] Satoshi NAKAMOTO. “Bitcoin : A Peer-to-Peer Electronic Cash System”. In : *Decentralized Whitepaper* (2008), p. 1–9.

- [19] Anil BEHAL, Jasminder Kaur SANDHU et Ganesh GUPTA. “Using The Cooja Simulator, Analysing The Routing Protocol (RPL) For Low Power And Lossy Networks In IoT”. In : *Proceedings of International Students’ Conference on Electrical, Electronics and Computer Science (SCEECS)*. IEEE. Bhopal, India, 2023, p. 1-4. DOI : 10.1109/SCEECS57921.2023.10061823.
- [20] PYTHON. *py-solc-x 2.0.5*. <https://pypi.org/project/py-solc-x/>. Accessed : April 20, 2026. 2025.
- [21] Yang LIU et al. “An overview of blockchain smart contract execution mechanism”. In : *Journal of Industrial Information Integration* 41 (2024), p. 100674. DOI : 10.1016/j.jii.2024.100674.
- [22] Goli ARCHANA, Rajeev GOYAL et K. KUMAR. “Blockchain-Driven Optimized Chaotic Encryption Scheme for Medical Image Transmission in IoT-Edge Environment”. In : *International Journal of Computational Intelligence Systems* 18.1 (2025), p. 1-24. DOI : 10.1007/s44196-024-00731-1.